Concept board

DIFFPLAYERQC

Audio workspace UI

CHALLENGE

Permitir QC de audio diferencial entre dos masters: visualizar waveforms A y B alineados, una pista de diferencia |A-B| que resalte regiones con discrepancias, y métricas EBU R128 (LUFS, true peak) por canal.

CORE IDEA

Stack vertical de tres tracks (A en verde, B en azul, diff en rojo), playhead naranja sincronizado, marcadores In/Out compartidos, regiones de diff resaltadas en rojo translúcido sobre los tres tracks, panel derecho con lista clicable de regiones detectadas y botón de auto-align.

EXECUTION

Backend Rust calcula peaks multi-resolución cacheados, loudness con crate ebur128 y detecta diff regions con ventana deslizante de 50ms. Frontend Svelte renderiza con SVG y sincroniza scroll/zoom con el reproductor.

Text

DiffPlayerQC v2 — Especificación técnica de migración a Tauri + Svelte

DiffPlayerQC v2 — Especificación técnica completa

Documento de implementación para agente de programación. Versión objetivo: **2.0.0**. Base actual: 1.5.0 (Tauri + vgui + wgpu + FFmpeg). Stack destino: **Tauri 2 + Svelte 5 + TypeScript + Rust (sin vgui)**.

0. Cómo leer este documento

- Las secciones marcadas **[MUST]** son requisitos no negociables.
- Las marcadas **[SHOULD]** son recomendadas pero opcionales.
- Las marcadas **[DON'T]** son anti-patrones.
- Cada bloque **MD-NOT** tiene **criterios de aceptación** al siguiente en avance al siguiente hasta que pasen todos.
- Los mockups visuales de referencia están en los artefactos `DiffPlayerQC_v2_Mockups.jsx` y `DiffPlayerQC_v2_Audio.mockups.jsx`. Las decisiones visuales y de paleta tienen precedencia sobre cualquier descripción textual.

1. Visión y objetivos

1.1 Producto

Reproductor diferencial de video y audio para QC profesional (broadcast / VFX / postpro). Compara dos fuentes A y B con precisión de fotograma y de muestra de audio, detecta discrepancias y produce informes.

1.2 Por qué v2

La v1 fue forzada a arrastrar todo el sistema de theming. Necesitamos:

- UI densa tipo DaVinci Resolve (paleta industrial, tipografía condensada, line-icons 1.5px).
- **Workspaces** intercambiables (Compare / Inspect / Audio / Report / Export).
- Paneles plegables con persistencia.
- Soporte de **audio diferencial** completo (no existía en v1).
- Soporte futuro para waveforms de audio, scopes de video (histograma, vectorscope, parade) y export a PDF.

1.3 No-objetivos de v2

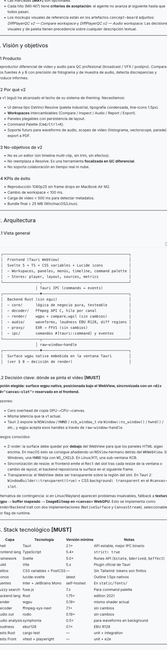
- No es un editor (no timeline multi-clip, sin efectos).
- No soporta **streaming** ni fuentes remotas — **solo archivos en disco**.
- No soporta colaboración en tiempo real ni nube.

1.4 KPIs de éxito

- Reproducción 1080p30 sin frame drops en MacBook Air M2.
- Cambio de workspace < 100 ms.
- Carga de video < 500 ms para detectar metadatos.
- Bundle final < 25 MB (Windows/Linux).

2. Arquitectura

2.1 Vista general



2.2 Decisión clave: dónde se pinta el video [MUST]

Opción elegida: surface wgpu nativa, posicionada bajo el WebView, sincronizada con un rAF/`ResizeObserver`.

Razones:

- Cero overhead de copia GPU→CPU→canvas.
- Misma latencia que la v1 actual.
- Tauri 2 soporta `raw-window-handle` (`HasRawWindowHandle`) en `WebviewWindow`.

Riesgos conocidos:

- Z-order: la surface debe quedar por **debajo** del WebView para que los paneles HTML sigan encima. En macOS esto se consigue añadiendo el NSView como hermano detrás del WebView (`addSubview:positioned:relativeTo:`). En Linux/X11 es trivial; en Wayland puede requerir ajustes.
- Sincronización de escala: el frontend debe enviar `devicePixelRatio` y tamaño lógico cada vez que cambie (`ResizeObserver` + `window.resize`).
- Transparencia: el WebView debe tener `transparent: true` y no pintar fondo opaco en `body`.
- Windows/DirectX: `transparent(true)` + CSS `background: transparent` en `#canvas-container`.

Alternativa de contingencia: si en Linux/Wayland aparecen problemas insalvables, fallback a **texture sharing** vía `SharedArrayBuffer` + `ImageBitmap` (rendimiento ~80% del nativo). Implementar solo si el bloque anterior no funciona.

2.3 Stack tecnológico [MUST]

Capa	Tecnología	Versión mínima	Notas
Shell	Tauri	2.x	API estable, MSRV 1.77+
Frontend lang	TypeScript	5.4+	strict: true
Framework	Svelte	5.x	Runes API (`$state`, `$derived`, `$effect`)
Build	Vite	5.x	Plugin oficial de Tauri
Estilos	CSS variables + PostCSS	—	Sin Tailwind; tokens en :root
Iconos	lucide-svelte	latest	Outline 1.5px native
Fuentes	Inter + JetBrains Mono	self-hosted	En static/fonts/
Testing	Vitest + Playwright	1.x	Para command palette
Backend lang	Rust	1.77+	sin cambios
Async	tokio	1.x	sin cambios
Decoder	ffmpeg-sys-next	7+	sin cambios
Audio	cpal + rubberband	0.15+	nuevo en v2
Audio analysis	symphonia	0.5+	para waveforms en background
Serialización	serde	1.x	JSON IPC
Tests Rust	cargo test	—	unit + integration
Tests Svelte	vitest + playwright	—	unit + e2e

4. Estructura de proyecto [MUST]

```
diffplayerqc/
├── src-tauri/                      # Backend Rust
│   ├── Cargo.toml
│   ├── tauri.conf.json
│   ├── build.rs
│   └── src/
│       ├── main.rs                 # entry: tauri::Builder, registra commands
│       ├── lib.rs                  # re-exports
│       ├── core/                   # lógica pura, sin Tauri ni vgui
│       │   ├── mod.rs
│       │   ├── player.rs           # PlayerState, PlaybackState
│       │   ├── compare.rs          # CompareMode, DiffMode
│       │   ├── markers.rs          # DvTool, regiones
│       │   └── timecode.rs         # parse/format SMPTE
│       ├── decoder/                # ex- decoder.rs*, refactor a módulo
│       │   ├── mod.rs
│       │   ├── video.rs
│       │   └── audio.rs
│       ├── proxy.rs
│       └── ipc/                    # ex- renderer.rs* + shader
```

Regla de oro [MUST]: el módulo `core/` no depende de `tauri`, `wgpu`, ffmpeg ni `cpal`. Es Pure Rust, testeable con `cargo test` sin features.

5. Sistema de diseño [MUST]

5.1 Tokens (en `src/lib/design/tokens.css`)

```
:root {
  /* Fondos */
  --bg-deep:#0a0c0f; --bg-base:#0d0f12;
  --bg-app:    #141414;
  --bg-panel:  #1a1a1a;
  --bg-raised: #222222;
  --bg-hover:  #2a2a2a;
  --border:    #2f2f2f;

  /* Acentos */
  --accent-primary:   #3498FE;
  --accent-primary-hover:#4EABFE;
  --accent-secondary: #E67E22;
  --accent-ok:        #2FD97F;
  --accent-warn:      #F1C40F;
  --accent-err:       #E63C41;

  /* Canales */
  --chan-a: #2F8FDF;
  --chan-b: #E67E22;
}
```

5.2 Componentes mínimos (`src/lib/components/control/`)

- `IconBtn.svelte` — 28×28, transparente, hover→bg-hover, active→accent-primary.
- `Segmented.svelte` — `{ options: {id,label}[], value, onchange }`. Usado en workspaces.
- `Slider.svelte` — `bipolar` (0 en centro) o `unipolar`, doble-click resetea.
- `Meter.svelte` — `{ value, peak, clip }`, verde→amarillo→rojo, clip rojo `#FF0000`.
- `Timecode.svelte` — `{ frames, fps, dropFrame }`, editable a la derecha, focus→accent.

5.3 Iconos

- Line outline/custom 1.5px. Tamaño 16px en toolbar, 14px en workspace tabs.
- **Sin gradientes**, sólo el playhead de timeline (gradiente vertical sutil de accent).
- **Sin sombras** salvo en modales y command palette.
- **Border radius máximo 6px.** Nada de pills.
- **Mayúsculas + letter-spacing 0.5px** en títulos de paneles y labels técnicos.
- **Nunca para todo sans-serif.** Inter/Lucide, frame, dB, LUFS, etc. SIN fallback.

6. Workspaces [MUST]

Los workspaces son **layouts predefinidos** que cambian la composición de paneles y el contenido principal. Cada uno se identifica por un id estable.

ID	Atajo	Estado v2	Layout
`compare`	`1`	Implementado	Source/Compare/Scopes + Inspector + Timeline
`inspect`	`2`	Implementado	Source/Compare + Inspector + Timeline
`audio`	`3`	**Nuevo en v2**	Source/Compare + Audio Channels + Timeline
`report`	`4`	Implementado	Source/Compare + Report + Timeline
`export`	`5`	Implementado	Source/Compare + Export + Timeline

6.1 Reglas de workspaces

- El workspace activo se persiste en `localStorage` y `backend` (`config.json`).
- Cada workspace puede **ocultar/mostrar paneles individuales** por workspace. El agente nunca debe usar un único objeto global de layout.
- Las transiciones entre workspaces son instantáneas (sin animación de transición). El cambio de paneles internos sí usa `grid-template-columns` con `transition: 160ms ease`.

7. Protocolo IPC [MUST]

Documentar en `docs/IPC_PROTOCOL.md`. Toda llamada (`invoke`) o todo evento debe estar listado allí. Generar tipos TypeScript desde Rust con `ts-rs` (`cargo install ts-rs`) o `specta`.

7.1 Comandos (frontend → backend)

```
// src-tauri/src/ipc/commands.rs

#[tauri::command]
pub async fn open_source(channel: Channel, path: String) -> Result<SourceInfo, String>

#[tauri::command]
pub async fn close_source(channel: Channel) -> Result<(), String>

#[tauri::command]
pub async fn play() -> Result<(), IpcError>

#[tauri::command]
pub async fn pause() -> Result<(), IpcError>

#[tauri::command]
pub async fn seek(frame: i64) -> Result<(), IpcError>

#[tauri::command]
pub async fn step(delta: i32) -> Result<(), IpcError>

#[tauri::command]
pub async fn set_compare(mode: CompareMode) -> Result<(), IpcError>
```

7.2 Eventos (backend → frontend)

Nombres cross string namespaced. Constantes en `src-tauri/src/ipc/events.rs` y en `src/lib/ipc/events.ts`.

Evento	Payload	Frecuencia	Descripción
`playback:tick`	`{ frame: i64, pts_ms: f64 }`	~60 Hz si reproduce	Sincronización de UI
`playback:state`	`{ playing: bool }`	Cambio	Play/pause
`source:opened`	`{ channel, info: SourceInfo }`	Una vez	Metadatos cargados
`source:error`	`{ channel, message }`	Una vez	Error de decode
`audio:level`	`{ ch: u32, rms: f32, peak: f32 }`	~30 Hz	Meters
`audio:clip`	`{ ch: u32, frame: i64 }`	Evento	Clip detectado
`proxy:ready`	`{ job_id, path }`	Una vez	Proxy generado
`proxy:progress`	`{ job_id, progress: f32 }`	Cada 500 ms	Progreso de proxy
`error:toast`	`{ level, message, source }`	Una vez	Error genérico
`error:live`	`{ title, message, severity }`	Una vez	Error persistente
`update`	`{ version, date, delta_mb, notes }`	Una vez	Update disponible

7.3 Errores

```
#[derive(Serialize, Clone, Debug)]
#[serde(tag="kind", content="data")]
pub struct IpcError {
    pub code: IpcErrorCode,
    pub message: String,
    pub details: Option<String>,
}

#[derive(Serialize, Clone, Debug)]
#[serde(rename_all="snake_case")]
pub enum IpcErrorCode {
    SourceNotOpen,
    InvalidPath,
    UnsupportedFormat,
    DecodeFailed,
    RendererError,
    AudioError,
    Io,
    Internal,
}
```

Frontend trata `IpcError` con un toast no bloqueante salvo `Internal` y `RendererError`, que abren modal.

8. Render de video: integración wgpu + Tauri [MUST]

8.1 Arquitectura de la surface

1. Al arrancar, el backend crea la `wgpu::Instance` y guarda referencia a la `tauri::WebviewWindow`.
2. El frontend monta un `<div id="canvas-slot">` y en cada `resize`/`scroll` envía `{x, y, w, h}` (coordenadas en píxeles físicos) vía `invoke('set_canvas_slot', ...)`.
3. El backend crea/reposiciona la surface wgpu para que coincida exactamente con ese rectángulo.
4. Si `w == 0 || h == 0` (panel colapsado), la surface se oculta (`set_visible(false)`).

8.2 Ciclo de frame

- Crea o reposiciona una sub-vista nativa hija de la ventana Tauri en esa región.
- Usa `wgpu::SurfaceConfiguration` con `present_mode: Fifo` (vsync).
- El WebView debe ser transparente sólo en esa región (`transparent: true` + `background: transparent` en `body`).

8.3 Fallback

El bloque de render de backend pinta a una surface wgpu el área de los frames decodificados (igual que v1).

8.4 Por plataforma

- **macOS**: `NSView` hermano detrás, **antes** del WebView (si no la primera). Usar `objc2` o `cocoa`.
- **Windows**: `HWND` hijo con `WS_CLIPSIBLING`. Usar `windows-rs`.
- **Linux/X11**: `XCreateSubwindow`. **Wayland**: usar vgui con `xdg_surface` separada y subsurface. Si esto da problemas, activar fallback canvas (§ 2.2).

8.5 Sincronización

`set_canvas_slot` debe **debounce** el frontend a 16 ms. El backend recrea la `SurfaceConfiguration` solo si cambia el tamaño en píxeles físicos (no si solo cambian las coordenadas).

8.6 Shader

Mismo `.wgsl` que la versión 1.5. Los uniforms se siguen escribiendo en el `tick` de render. El frontend solo maneja comandos de alto nivel; nunca toca uniforms directamente.

8.7 Tests de integración

- Abrir dos archivos de distinta resolución y verificar que el shader escala correctamente.
- Cambiar de workspace 100 veces seguidas sin leaks de surface (monitorizar con `Instruments` / `heaptrack`).
- Redimensionar la ventana a 1×1 y a 8K sin crash.

9. Audio: pipeline nuevo [MUST]

9.1 Pipeline

Ffmpeg decoder → f32 PCM stereo @ 48kHz
Ring buffer (SPSC, lock-free)
cpal output stream
Tap: FFT + RMS + peak
Rubberband (time-stretch)
Tap: waveform + LUFS
Tap: vectorscope + correlation
Tap: FFT + RMS + peak
Tap: waveform + LUFS
Tap: vectorscope + correlation
Mixer: A / B / Diff / Mono
Tap: FFT + RMS + peak
Tap: waveform + LUFS
Tap: vectorscope + correlation

`Shift+←` / `Shift+→`	Saltar 1 segundo
`Home` / `End`	Inicio / Fin
`I` / `O`	Mark In / Out
`Shift+I` / `Shift+O`	Clear In / Out
`1`–`5`	Modo de comparación
`Y`	Cycle compare mode
`S`	Swap A/B
`F`	Screenshot
`R`	Reset zoom/pan
`+` / `-`	Zoom in/out
`Cmd/Ctrl+1..5`	Toggle paneles
`Cmd/Ctrl+I` / `Cmd/Ctrl+J`	Toggle panel izq/der
`Cmd/Ctrl+K`	Command Palette
`Shift+1..5`	Cambiar workspace
`Cmd/Ctrl+Shift+F`	Full screen
`Esc`	Cerrar modal / palette

Cuando el foco está en un `<input>`, **solo** funcionan: `Esc`, `Cmd/Ctrl+K`, `Cmd/Ctrl+S`.

---

12. Command Palette **[MUST]**

- Componente `CommandPalette.svelte` montado en `App.svelte`, oculto por defecto.
- Lista de comandos en `src/data/slicecommands.ts`: cada entrada `{ id, label, category, shortcut?, run: () => void }`.
- Búsqueda con fuse.js sobre `label` + `category`.
- Categorías: `Workspace`, `Playback`, `View`, `Compare`, `Audio`, `Export`, `Help`.
- Mostrar máximo 8 resultados. Navegación con flechas y Enter.

---

13. Persistencia y configuración **[MUST]**

13.1 Archivo de config

```
directories::ProjectDirs::from("com", "dlfplayerqrc", "dlfplayerqrc").config_dir()/config.json
```

```rust
pub struct AppConfig {
  pub language: Language,
  pub theme: Theme, // siempre "dark-resolve" en v2; conservar enum por compat
  pub last_workspace: WorkspaceId,
  pub layouts: HashMap<WorkspaceId, WorkspaceLayout>,
  pub player_defaults: PlayerDefaults,
  pub screenshot_dir: Option<PathBuf>,
  pub canvas_bg: [f32; 3],
  pub audio: AudioConfig,
  pub recent_files: VecDeque<File>, // máx 10
}
```

13.2 Escritura atómica

Mantener el patrón v1 (`config.json.tmp` + rename). Frontend nunca toca el archivo; usa `save_config`/`load_config`.

13.3 Idiomas

v2 elimina Quenya. Solo Es y En. Strings en `src/lib/i18n/{es,en}.ts`. Función `t(key)` reactiva al cambio de idioma.

---

14. Migración desde v1 (egui) **[MUST]**

Fase A — Preparación (sin tocar UI v1)

1. **Refactor del actual app/`mod.rs`**: extraer toda la lógica no-UI a `core/` puro. La v1 sigue compilando contra ese `core/`.
2. **Tests de regresión**: ampliar la cobertura unitaria de `core/` (player state machine, seek, mode cycling, marker logic).
3. **Eliminar dependencias muertas**: `dark-light` (sin uso real en v2 oscuro), strings Quenya.

Fase B — Tauri shell

1. Crear `src-tauri/` con Tauri 2 init. Reusar `Cargo.toml` deps de FFmpeg/wgpu.
2. Mover `core/`, `decoder/`, `render/`, `proxy/` al backend Tauri.
3. Implementar comandos IPC mínimos para `open_source`, `play`, `pause`, `seek`. UI temporal con HTML estático.

Fase C — UI Svelte

1. Layout shell (menubar, toolbar, statusbar, workspace tabs).
2. Componente `Panel` y todos los controles base.
3. Workspace `Compare` funcional (sin canvas todavía).

Fase D — Render wgpu nativo

1. Implementar `set_canvas_slot` y posicionamiento de `NSView`/`HWND` hijo.
2. Conectar render loop existente a la nueva surface.
3. Validar en macOS, Windows, Linux X11. Wayland si hay tiempo.

Fase E — Audio nuevo

1. `audio_analysis/` con waveform builder.
2. Workspace `Audio` con waveforms A, B y diff strip.
3. Loudness EBU R128.

Fase F — Acabado

1. Command palette.
2. Atajos completos.
3. Inspect workspace con scopes (histogram + vectorscope + waveform monitor).
4. Persistencia por workspace.
5. Tests e2e.

Decomisado de v1

La rama `egui-legacy` se conserva en git pero deja de recibir features. Bugfixes críticos de seguridad solamente.

---

15. Hitos y criterios de aceptación **[MUST]**

M0 — Bootstrap (1 sem)

Entregables: repo Tauri 2 + Svelte 5 compila. CI verde. `core/` extraído de v1. **Aceptación:**

- `cargo build && pnpm build` funcionan en macOS, Windows, Linux.
- `cargo test -p core` con cobertura ≥ 70% sobre `core/`.
- CI workflow `.github/workflows/ci.yml` corre fmt + clippy + tests + `pnpm test`.

M1 — Apertura y reproducción (1–2 sem)

Entregables: abrir A y B, reproducir, seek, step, sin canvas (placeholder negro). **Aceptación:**

- Cargar 4 archivos de test (mp4, mov ProRes, mkv h264, mxf) sin error.
- Eventos `playback://tick` llegan a 60 Hz en reproducción.
- Reproducción fluida con audio (todo) sin underruns 60s seguidos.

M2 — UI shell completa (2 sem)

Entregables: menubar, toolbar, paneles colapsables, workspace tabs, command palette, atajos. **Aceptación:**

- Mockups Compare y Audio reproducidos visualmente con desviación pixel a 2%.
- Cambio de workspace < 100 ms (medido con Performance API).
- Command palette abre con `Cmd/Ctrl+K`, navegación funcional.
- Tests Playwright: abrir cada workspace y verificar layout.

M3 — Render wgpu (1–2 sem)

Entregables: video se ve en el `<div id="canvas-slot">` del workspace Compare. **Aceptación:**

- 1080p25 sin frame drops en MacBook Air M2 (medido sobre 60s).
- Resize de la ventana redibuja correctamente sin glitches en macOS y Windows.
- Modos Split, Diff, Heatmap, SideBySide funcionan idénticos a v1.

M4 — Audio nuevo (2 sem)

Entregables: workspace Audio con waveforms A/B + diff strip + LUFS. **Aceptación:**

- Waveform 1080px de un mp4 de 5 min generado en < 3 s.
- Click en una región de diff salta el playhead a ese PTS.
- LUFS calculados coinciden con `ffmpeg -af ebur128` (margen ±0.1 LU).

M5 — Inspect + scopes (2 sem)

Entregables: histogram RGB, vectorscope, waveform monitor. **Aceptación:**

- Scopes se actualizan cada frame al reproducir sin caer fps.
- Implementados como wgpu compute o canvas 2D (libre elección).

M6 — Persistencia y polish (1 sem)

Entregables: layouts persistentes por workspace, idiomas Es/En, recent files. **Aceptación:**

- Cerrar y reabrir restaura último workspace y disposición exacta.
- `recent_files` muestra últimos 10, abrir desde menú File funciona.

M7 — Release (1 sem)

Entregables: build firmado para macOS (`.pkg`), Windows (`.zip` portable + `.msi`), Linux (`.AppImage` + `.deb`). **Aceptación:**

- Bundle final macOS < 25 MB.
- CI release publica artifacts en GitHub Releases automáticamente.

---

16. Tests **[MUST]**

16.1 Rust

- `core/` unit tests cubren state machine de playback, mode cycling, marker logic, layout persistence.
- `decoder/` integración con archivos de test en `tests/fixtures/` (10 segundos cada uno, varios codecs).
- `audio_analysis/`: golden tests contra valores conocidos (LUFS, peaks).

16.2 Frontend

- **Unit (vitest)**: stores, utilidades (timecode, format), command palette filter.
- **Component (vitest + @testing-library/svelte)**: cada componente de `controls/` y `panel/`.
- **E2E (Playwright)**: flujos completos
- Abrir A y B, reproducir 5s, hacer screenshot.
- Cambiar a Audio workspace, click en una diff region, verificar seek.
- Command palette: escribir "audio", verificar primer resultado.
- **Visual (Playwright + pixelmatch)**: una imagen por workspace en estado default.

16.3 CI

GitHub Actions, matrix (`ubuntu-latest`, `macos-latest`, `windows-latest`). Pasos: `cargo fmt --check`, `cargo clippy -- -D warnings`, `cargo test`, `pnpm test`, `pnpm e2e`. PRs no se mergean si algo falla.

---

17. Logging y telemetría **[SHOULD]**

- Mantener `trace_log.rs` de v1: un fichero por sesión.
- Añadir niveles: `TRACE`/`DEBUG`/`INFO`/`WARN`/`ERROR`.
- En frontend: capturar errores con `window.onerror` y reenviarlos al backend vía comando `report_frontend_error`.
- **Sin telemetría externa**. Nada sale del equipo del usuario.

---

18. Accesibilidad **[SHOULD]**

- Contraste AA mínimo (la paleta actual lo cumple para texto principal sobre `--bg-panel`).
- Focus visible en todos los controles: outline `2px --accent-primary`.
- Atributos `aria-label` en todos los icon buttons.
- Soporte de tab navigation en orden lógico.
- No hay WCAG total como objetivo: es herramienta profesional de QC, no de cara al público general.

---

19. Build y distribución **[MUST]**

19.1 Dev

```bash
pnpm install
pnpm tauri dev
```

19.2 Producción

```bash
pnpm tauri build
```

Genera: `.dmg`/`.app` en macOS, `.msi`/`.exe` en Windows, `.AppImage`/`.deb` en Linux.

19.3 Firma

- **macOS**: notarización con Developer ID. Clave en GitHub Secrets.
- **Windows**: Authenticode signing. Certificado en GitHub Secrets.
- **Linux**: sin firma.

19.4 Auto-update

[MAY]: usar `tauri-plugin-updater`. Backend de updates puede ser GitHub Releases mismo. No bloqueante para v2.0.0.

---

20. Seguridad **[MUST]**

- `tauri.conf.json` con allowlist mínima: solo los comandos definidos en `ipc/commands.rs` expuestos.
- CSP estricta en producción: `default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'`.
- `unsafe-inline` solo para Svelte styles.
- Validar todas las paths recibidas por `open_source` (canonicalización + check de existencia).
- Sin `shell.open` ni `dialog.open` de Tauri salvo donde estrictamente necesario (file dialogs nativos).

---

21. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Wayland + wgpu surface child no funciona	Media	Alto	Fallback canvas (§ 2.2). Decidir en M3.
Bundle final supera 25 MB	Baja	Medio	strip + lto + sin Tailwind. Aceptar 30 MB si imprescindible.
Rescribir UI atrasa la entrega	Alta	Alto	Hitos cortos, mantener v1 publicable hasta M4.
EBU R128 calcula distinto a Resolve	Media	Bajo	Validar contra ffmpeg, no contra Resolve (R128 es estándar).
WebView en Windows (WebView2) ausente en Win10 viejos	Baja	Medio	Bootstrap instalar de WebView2 incluido en MSI.

---

22. Glosario

- **PTS**: presentation timestamp, en segundos.
- **LUFS / LU**: Loudness Units (Full Scale / relative). EBU R128.
- **Diff region**: tramo temporal donde la diferencia entre A y B supera un umbral.
- **Workspace**: layout predefinido con disposición de paneles y contenido central específico.
- **Slot**: `<div>` reservado en el HTML que define dónde se posiciona la surface wgpu nativa.

---

23. Referencias

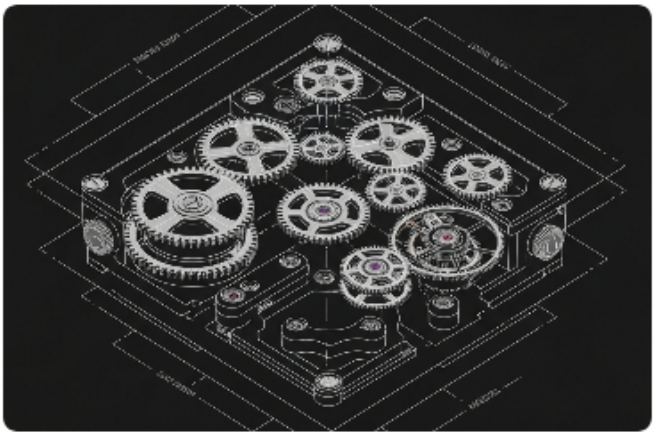
- Repo legacy v1 (egui): `egui-legacy` branch.
- Mockups visuales: artefactos `concept-board` *DiffPlayerQC v2 — Compare workspace* y *DiffPlayerQC v2 — Audio workspace* en este chat.
- Paleta y tipografía: `docs/DESIGN_SYSTEM.md` (a generar en M0 a partir de § 5 de este doc).
- IPC: `docs/IPC_PROTOCOL.md` (a generar en M1 a partir de § 7).
- Tauri 2 API: https://v2.tauri.app/
- Svelte 5 runes: https://svelte.dev/docs/svelte/what-are-runes
- wgpu + Tauri integración: `clearlysid/tauri-wgpu-can` (referencia de patrón).

---

24. Checklist final para el agente antes de empezar

- [ ] Confirmado: existe rama `egui-legacy` con la v1 intacta.
- [ ] Confirmado: este documento está en `docs/SPEC_v2.md`.
- [ ] Mockups visuales descargados a `docs/design/` como referencia.
- [ ] Ramas de feature por hito: `m0/bootstrap`, `m1/playback`, etc.
- [ ] PR template creada con sección "Acceptance criteria covered".
- [ ] Issue de tracking abierta por hito enlazando a § 15 de este doc.

Thought

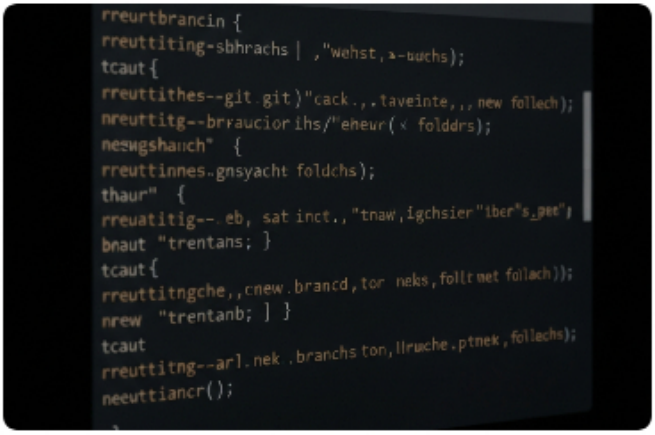


EXTRAER CORE/ DE LA V1

Refactor de `src/app/mod.rs` v1: mover `PlayerState`, `CompareMode`, `DiffMode`, `WorkspaceLayout` a un módulo `core/` puro sin dependencias de egui ni wgpu. La v1 sigue compilando contra ese core. Cobertura unitaria ≥ 70%.

Prerrequisito limpio para Tauri sin reescribir lógica.

Thought

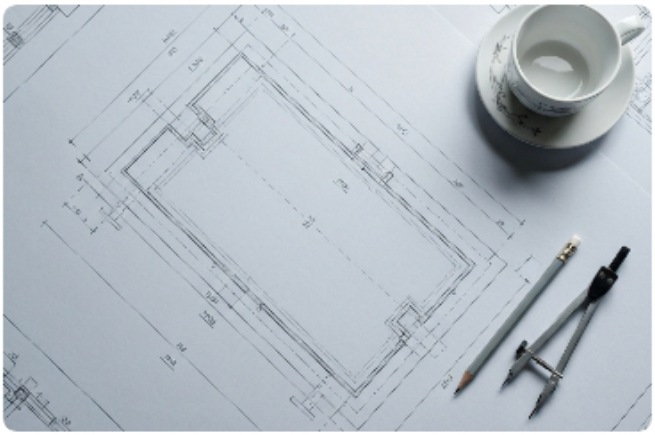


GENERAR EL SPEC EN EL REPO

Crear rama `m0/bootstrap`, copiar este documento a `docs/SPEC_V2.md`, abrir issue raíz "Migración v2" enlazando a la sección 15 (hitos).

Setup inicial antes de tocar código.

Thought



SKELETON TAURI + SVELTE

`pnpm create tauri-app` con template Svelte 5 + TS. Configurar Vite, ESLint, Prettier, Vitest, Playwright. Migrar `Cargo.toml` deps de FFmpeg/wgpu al nuevo `src-tauri/`. CI verde antes de cualquier feature.

Hito M0 completo.